

Tech Stacks to Achieve High Availability

 systemdesignschool.io/fundamentals/tech-stacks-to-achieve-high-availability



Let's take a look at the tech frameworks and stacks we can use to achieve high availability. This section is "good to know" information. In a real interview, you wouldn't be expected to dive too deep into specific technologies but it's nice to know the tech stacks so you are not just arm waving.

Let's start with open-source options:

Open Source

[HAProxy](#)

This is a free, open-source software that provides a high availability load balancer and proxy server for TCP and HTTP-based applications. It's used by many high-profile businesses handling significant levels of web traffic (like Twitter, Airbnb, and GitHub) to improve the performance and reliability of their servers.

HAProxy distributes the workload across multiple servers to optimize resource usage, maximize throughput, minimize response time, and avoid overloading any single server. It also provides automatic failover, meaning if one of your servers goes down, HAProxy will automatically redirect traffic to the remaining operational servers.

[Keepalived](#)

This is another open-source software that provides simple and robust facilities for load balancing and high availability. The load balancing framework relies on the well-known and widely used Linux Virtual Server (IPVS) for network-based load balancing, while the high availability component is based on VRRP protocol.

Keepalived also has health-checking mechanisms that can help monitor real servers and, in case of failure, it can decide to change the LVS topology and remove or add real servers.

Now let's take a look at the big 3 cloud providers and their HA services:

Amazon Web Services (AWS)

1. [Elastic Load Balancer \(ELB\)](#): This service automatically distributes incoming application traffic across multiple targets, such as Amazon EC2 instances. It improves the fault tolerance of your applications as the traffic is spread across several resources.
2. [Amazon RDS \(Relational Database Service\)](#): The Multi-AZ deployments option for the RDS enables you to run mission-critical databases with high availability and built-in automated fail-over from your primary database to a synchronously replicated secondary database in case of a database instance failure.
3. [Amazon EC2 Auto Scaling](#): This service helps ensure that you have the correct number of Amazon EC2 instances available to handle the load for your application. In combination with ELB, Auto Scaling enables your applications to handle increases in traffic and withstand failures of an instance or availability zone.

Google Cloud Platform (GCP):

1. [Cloud Load Balancing](#): A fully distributed, software-defined, managed service for all your traffic. It offers autoscaling, and it's not just a load balancer but also an intelligent traffic distributor, with cross-region load balancing including automatic multi-region failover which steers your traffic to available healthy instances across multiple regions.
2. [Cloud SQL](#): This is a fully-managed database service that makes it easy to set up, maintain, manage, and administer your relational databases on Google Cloud Platform. It offers high availability configuration to protect your database from zonal failures.
3. [Managed Instance Groups \(MIGs\)](#): These ensure that your deployed services are distributed across multiple, isolated failure domains and can scale up or down with intelligence. This helps in auto-healing and maintaining high availability of your application.

Microsoft Azure:

1. [Azure Load Balancer](#): This built-in load balancing feature for Azure deployments can distribute traffic among similar systems, improving system responsiveness and availability.
2. [Azure SQL Database](#): It's a fully managed relational database service that offers SQL Server engine compatibility and built-in high availability solution. It uses a technology called Always On availability groups to ensure that a group of databases is available and to provide failover support for single or multiple databases.
3. [Azure Virtual Machine Scale Sets](#): This allows you to create and manage a group of identical, load balanced, and autoscaling VMs. The number of VM instances can automatically increase or decrease in response to demand or a defined schedule, offering high availability to your applications.

Availability vs Fault Tolerance

Fault tolerance and availability are two related concepts in system design, which together contribute to the reliability of a system. Here's how they relate:

Availability:

Availability, as we discussed, is a measure of the system's uptime. It refers to the time a system or a service is up and running, or available for use. It's usually defined as the percentage of time that a system is operational and able to provide service as expected.

Fault Tolerance:

Fault tolerance refers to the ability of a system to continue operating correctly even in the event of partial system failures. This might involve hardware or software redundancy, error handling, retry mechanisms, or self-healing processes.

In essence, a fault-tolerant system is designed to eliminate single points of failure. So, even if one component of the system fails, the system can continue functioning without interruption. Fault-tolerant systems are able to detect and repair faults automatically without human intervention.

By this definition, you can consider fault tolerance to mean 100% available, which isn't realistic.

The Relationship between Availability and Fault Tolerance:

Fault tolerance directly contributes to the system's high availability. You can consider high availability as the result and making the system fault-tolerant as one way of achieving it. By ensuring the system continues to operate correctly even when some components fail, fault tolerance prevents system-wide downtime and therefore increases the overall availability of the system.

For the purpose of system design interviews, fault tolerance and high availability can almost be considered identical concepts. When the interviewer asks for fault tolerance, they probably just mean availability.